DOCUMENTAÇÃO DE ENGENHARIA DE SOFTWARE & ARQUITETURA DE SEGURANÇA

MANUAL TÉCNICO DO SISTEMA SCRC v3.0

Arquitetura Full-Stack, Modelagem Relacional, Máquina de Estados, Deploy Serverless e Especificação das 20 Camadas de Segurança Defensiva (OWASP)

FICHA TÉCNICA DA ARQUITETURA

LINGUAGENS CORE
Python 3.12+ (Backend) • JavaScript ES2022, HTML5, CSS3 (Frontend SPA) • SQL (PostgreSQL 15)

FRAMEWORKS & ORM
FastAPI >= 0.115.0 • SQLAlchemy 2.0 ORM • Pydantic v2 • Uvicorn ASGI

BANCO DE DADOS PRODUÇÃO
Supabase PostgreSQL (Transaction Pooler IPv4 Supavisor :6543 via driver puro Python pg8000)

INFRAESTRUTURA & CI/CD
Vercel Serverless Functions + GitHub Integration (Deploy Contínuo na branch main)

CRIPTOGRAFIA & AUTENTICAÇÃO
PBKDF2-HMAC-SHA256 (200k iter) • Sessões HMAC-SHA256 • Cifra CTR-HMAC-SHA256 (Encrypt-then-MAC)

POSTURA DE SEGURANÇA
20/20 Controles Ativos (RLS PostgreSQL, CSP, HSTS, RBAC, Rate Limit 3 Níveis, Anti-CSRF, Anti-XSS)

Repositório Oficial: github.com/jeanjsj-beep/controle-cabecotes
URL de Produção: https://controle-cabecotes-hpef.vercel.app

Público-Alvo: Engenheiros de Software, Arquitetos e DevSecOps
Versão da Especificação: 3.0.0

1. Stack Tecnológica e Linguagens Utilizadas

ARQUITETURA DE SOFTWARE

O sistema **SCRC** foi projetado sob uma arquitetura **Full-Stack Stateless Serverless-First**, desacoplando a interface SPA (Single Page Application) da API RESTful em Python através de contratos JSON validados por schemas estritos. Essa escolha garante zero dependência de disco efêmero em nuvem e baixa latência tanto em redes locais quanto em execução distribuída na Vercel.

Camada	Linguagem / Tecnologia	Papel Arquitetural e Justificativa de Engenharia
Backend API	`Python 3.12+` `FastAPI >= 0.115.0`	Framework ASGI de alta performance baseado em Starlette, com injeção de dependências nativa (usada para sessões de banco e RBAC) e serialização assíncrona.
Validação de Dados	`Pydantic v2 >= 2.9.0`	Validação estrita de payloads de entrada/saída em Rust-core, prevenindo *Mass Assignment*, coerção indevida de tipos e payloads malformados.
ORM & Persistência	`SQLAlchemy >= 2.0.35`	Mapeamento Objeto-Relacional com queries 100% parametrizadas (proteção nativa contra SQL Injection) e suporte híbrido PostgreSQL / SQLite.
Driver de Banco	`pg8000 >= 1.31.2` `psycopg2-binary >= 2.9.9`	O driver `pg8000` (100% puro Python) garante compatibilidade total em containers AWS Lambda / Vercel Linux sem depender de bibliotecas C dinâmicas (`libpq`).
Banco em Nuvem	`Supabase PostgreSQL 15`	Banco relacional gerenciado com **RLS (Row-Level Security)** ativado nas 8 tabelas e conexão via Pooler IPv4 Supavisor (porta `6543`).
Frontend SPA	`JavaScript ES2022` `HTML5 + CSS3`	Aplicação SPA sem sobrecarga de bundlers pesados. Inclui interceptador global de `window.fetch` , compressão client-side de fotos via `<canvas>` e layout responsivo mobile/desktop.

2. Estrutura de Diretórios e Engenharia Serverless

ORGANIZAÇÃO DO CÓDIGO

```
controle_cabecotes/
├── vercel.json                    # Roteamento Serverless para Vercel (@vercel/python)
├── requirements.txt               # Dependências fixadas com versões seguras
├── backend/
│   ├── app/
│   │   ├── main.py                # Ponto de entrada FastAPI, Middlewares OWASP, Upload e Arquivos Estáti
cos
│   │   ├── database.py            # Engine SQLAlchemy, normalização pg8000/Pooler IPv4 e Auto-Migration
│   │   ├── models.py              # 8 Modelos ORM (Tabelas relacionais PostgreSQL/SQLite)
│   │   ├── schemas.py             # Contratos Pydantic v2 (Inputs/Outputs)
│   │   └── routers/
│   │       ├── auth.py            # Autenticação PBKDF2, Sessões HMAC, RBAC, RLS, Cifra CTR-HMAC, Backup
e Auditoria
│   │       ├── clientes.py        # CRUD de Clientes
│   │       ├── lotes.py           # Entrada unificada de Lotes, Recebimento Físico e Expedição/Pallets
│   │       ├── cabecotes.py       # Listagem de Cabeçotes e Definição de Rota (Checklist de Triagem)
│   │       ├── etapas.py          # Máquina de Estados das 8 Etapas da Oficina e Gerador de Laudo Técnico
JSON/PDF
│   │       └── dashboard.py       # Cálculo de OEE, SLA, Monitor de Gargalos e Ocupação de Pallets
└── frontend/
    ├── index.html                 # Estrutura SPA (14 telas operacionais + Painel Admin & 20 Checks + Mod
ais)
    ├── styles.css                 # Design System Glassmorphism, Responsividade Mobile (<768px) e @media
print
    └── app.js                     # Lógica de Estado, Fetch Interceptor (Bearer + CSRF), Compressão de Fo
tos e RBAC UI
```

2.1. Soluções de Engenharia para Ambientes Serverless (Vercel + Supabase)

- **Normalização Automática de `DATABASE_URL` em `database.py`:** Como funções Serverless na Vercel operam em rede IPv4 e o host direto do Supabase (`db.<ref>.supabase.co:5432`) utiliza IPv6 exclusivo, o módulo `database.py` reescreve automaticamente conexões diretas para o **Supabase Transaction Pooler IPv4** (`aws-1-sa-east-1.pooler.supabase.com:6543` com usuário `postgres.<ref>`) e converte o dialeto para `postgresql+pg8000://` com `NullPool`.
- **Armazenamento de Fotos no Próprio Banco (Stateless Storage):** Em ambientes Serverless o sistema de arquivos local é efêmero e somente-leitura. Por isso, o endpoint `POST /api/upload-foto` armazena o binário validado diretamente na tabela `fotos_armazenadas` (em Base64) e o serve com cache imutável em `GET /uploads/{filename}`.

3. Modelagem do Banco de Dados Relacional (8 Tabelas)

SCHEMA POSTGRESQL

O schema relacional é definido em `backend/app/models.py` e sincronizado automaticamente no boot por `Base.metadata.create_all()` combinado com migrações incrementais seguras (`ALTER TABLE ... ADD COLUMN`) em `check_and_migrate_db()`:

Tabela	Colunas Chave	Responsabilidade no Domínio
`usuarios`	`id (PK)`, `username (UQ)`, `senha_hash`, `senha_salt`, `is_admin`, `permissoes_json`, `tentativas_falhas`, `bloqueado_ate`	Credenciais de acesso com hash PBKDF2, matriz de permissões RBAC serializada em JSON e controle de bloqueio anti-força bruta.
`clientes`	`id (PK)`, `nome`, `cnpj`, `inscricao_estadual`, `email`, `telefone`	Cadastro fiscal e comercial dos proprietários dos lotes de cabeçotes.
`lotes`	`id (PK)`, `codigo_lote (UQ)`, `cliente_id (FK)`, `status`, `local_alocacao`, `expedicao_nf_retorno`, `expedicao_motorista`	Agrupador logístico das peças. Gerencia o ciclo macro (`RECEBIDO`, `EM_PROCESSAMENTO`, `CONCLUIDO`, `EXPEDIDO`) e o endereço de pallet (`ALM-A/B`).
`cabecotes`	`id (PK)`, `numero_serie (UQ)`, `lote_id (FK)`, `status_etapa`, `requer_*`, `estanqueidade_pressao`, `faceamento_medida_*`, `relatorio_individual_json`	Entidade central de rastreabilidade unitária. Guarda as flags de rota da triagem, métricas dimensionais de cada posto, fotos e o JSON completo do Relatório Técnico Individual.
`movimentacoes`	`id (PK)`, `cabecote_id (FK)`, `etapa`, `operador`, `data_inicio`, `data_fim`, `resultado`, `observacoes`	Log histórico de apontamentos de chão de fábrica por peça, usado para cálculo de lead-time, gargalos e emissão do laudo técnico.
`fotos_armazenadas`	`nome_arquivo (PK)`, `mime_type`, `conteudo_base64`, `criado_em`	Persistência binária em banco das evidências fotográficas capturadas nas etapas operacionais.
`auditoria_seguranca`	`id (PK)`, `data_hora`, `evento`, `severidade`, `usuario`, `ip_origem`, `detalhes`	Trilha de auditoria imutável (SIEM interno) para logins, falhas, bloqueios, alterações de permissão, rate limits e backups.
`backups_automaticos`	`id (PK)`, `criado_em`, `tipo`, `autor`, `sha256_checksum`, `payload_criptografado`	Armazena snapshots completos do banco cifrados com `CTR-HMAC-SHA256` e verificados por hash de integridade `SHA-256`.

4. Máquina de Estados Finitos (FSM) e Regras de Negócio

REGRAS DE DOMÍNIO

4.1. Roteamento Dinâmico de Etapas (`avancar_proxima_etapa`)

Em `backend/app/routers/etapas.py`, a transição entre postos não é rígida; ela consulta os atributos booleanos `requer_*` configurados na Triagem do cabeçote. A ordem canônica avaliada é:

```
ORDEM_ETAPAS = [
    ("LAVAGEM",       True),                       # Sempre executada após a Triagem
    ("JATEAMENTO",    cab.requer_jateamento),
    ("ESTANQUEIDADE", cab.requer_estanqueidade),
    ("FACEAMENTO",    cab.requer_faceamento),
    ("LAPIDACAO",     cab.requer_lapidacao),
    ("MONTAGEM",      cab.requer_montagem),
    ("QUALIDADE",     cab.requer_qualidade),       # Gera e persiste o Relatório Individual JSON/PDF
    ("PINTURA",       cab.requer_pintura),
] -> Finaliza em "PRONTO" (ou "CONDENADO" se reprovar na Estanqueidade)
```

4.2. Regra de Retrabalho em 2 Níveis na Lapidação & Vácuo

- **Aprovação (`APR`):** Limpa o status de retrabalho e avança a peça para `MONTAGEM`.
- **1ª Reprovação no Vácuo (`REP` com `tentativas == 1`):** Mantém o cabeçote na etapa `LAPIDACAO` e define `lapidacao_status_retrabalho = "TROCAR_VALVULA"`.
- **2ª+ Reprovação no Vácuo (`REP` com `tentativas >= 2`):** Mantém o cabeçote em `LAPIDACAO` e escala a exigência para `lapidacao_status_retrabalho = "TROCAR_SEDE"`.

5. Arquitetura de Segurança: Especificação das 20 Verificações DEVSECOPS & OWASP

O sistema implementa 20 camadas de proteção ativa no backend (`main.py` e `routers/auth.py`), no banco de dados (Supabase PostgreSQL) e no cliente SPA (`app.js`):

#	Controle de Segurança	Implementação Técnica e Mecanismo de Proteção
01	**HTTPS & HSTS**	Middleware inspeciona `X-Forwarded-Proto` em produção e força redirecionamento `307 Temporary Redirect` para `https://` , além de injetar `Strict-Transport-Security: max-age=63072000; includeSubDomains; preload` .
02	**CORS Restrito**	`CORSMiddleware` configurado com lista explícita de origens confiáveis (produção Vercel, localhost e `SCRC_CORS_ORIGINS`), restringindo métodos e cabeçalhos permitidos.
03	**Senhas com Hash**	Derivação `hashlib.pbkdf2_hmac('sha256', password, salt, 200000)` com salt aleatório de 16 bytes (128 bits) por usuário e verificação em tempo constante via `hmac.compare_digest()` . Política exige 8+ caracteres, maiúscula, minúscula, dígito e símbolo.
04	**Cookies Seguros**	No login, o servidor emite o cookie `scrc_session` com flags `HttpOnly; Secure; SameSite=Strict; Path=/; Max-Age=86400` , impedindo leitura via JavaScript malicioso.
05	**Proteção de Secrets**	Chaves de assinatura e criptografia derivadas via KDF (PBKDF2) a partir de variáveis de ambiente (`SCRC_AUTH_SECRET` , `SCRC_ENCRYPTION_KEY`). Documentação `/docs` , `/redoc` e `/openapi.json` desativadas por padrão em produção.
06	**Controle de Acesso (RBAC)**	Dependências FastAPI `requer_permissao(modulo)` e `requer_admin` aplicadas em todas as rotas de negócio. Tentativas de acesso sem permissão geram HTTP 403 e alerta na tabela `auditoria_seguranca` .
07	**Gestão de Sessões**	Tokens stateless assinados no formato `s2.<uid>.<token_version>.<exp>.<nonce>.<hmac_sig>` transmitidos exclusivamente via cookies `HttpOnly; Secure; SameSite=Strict` (sem armazenamento em LocalStorage/SessionStorage), controle de versão de sessão (`token_version`) para revogação instantânea em logout e política de troca obrigatória de senha (`must_change_password`).
08	**RLS (Row-Level Security)**	Função `aplicar_rls_postgresql()` executa `ALTER TABLE public.<t> ENABLE ROW LEVEL SECURITY` e `REVOKE ALL ON TABLE public.<t> FROM anon, authenticated` nas 8 tabelas do Supabase, bloqueando acesso direto pela API PostgREST pública.
09	**Rate Limits (3 Níveis)**	Controle por IP em janela deslizante: **Login** (máx. 15 req/min), **Uploads** (máx. 20 req/min) e **API Global** (máx. 120 req/min), retornando `HTTP 429` + bloqueio de conta por 15 min após 5 falhas de senha.
10	**Validação de Inputs**	Schemas Pydantic v2 estritos combinados com validação de `Content-Length` no middleware (bloqueio automático `HTTP 413` acima de 15 MB).
11	**Anti-SQL Injection**	Uso exclusivo de queries parametrizadas do SQLAlchemy ORM em todos os routers. Nenhum input de usuário é concatenado em strings SQL.
12	**Proteção Anti-XSS**	Sanitização server-side via `sanitizar_texto_seguro()` (remoção de tags HTML, caracteres de controle e atributos `javascript:/on*=`) + função `escapeHtml()` no frontend + cabeçalho `Content-Security-Policy` .
13	**Proteção Anti-CSRF**	Defesa em profundidade: cookies `SameSite=Strict` + validação de igualdade entre cabeçalho `Origin` e `Host` em métodos `POST/PUT/DELETE/PATCH` + envio automático de `X-Requested-With: XMLHttpRequest` e `X-CSRF-Token` .

#	Controle de Segurança	Implementação Técnica e Mecanismo de Proteção
14	**Security Headers**	Injeção global de `Content-Security-Policy`, `Strict-Transport-Security`, `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Referrer-Policy`, `Permissions-Policy` e `Cross-Origin-Opener-Policy: same-origin`.
15	**Uploads Seguros**	Inspeção dos **Magic Bytes** reais do binário decodificado (assinaturas `\xFF\xD8\xFF` JPEG, `\x89PNG`, `RIFF..WEBP`, `GIF8`), teto de 10 MB, renomeação com UUIDv4 e entrega com `nosniff`.
16	**Tratamento de Erros**	Handler global `@app.exception_handler(Exception)` que captura falhas inesperadas, grava o erro internamente com um `trace_id` único e retorna mensagem genérica segura sem expor stack traces.
17	**Criptografia de Dados**	Implementação de cifra de fluxo autenticada **CTR-HMAC-SHA256 (Encrypt-then-MAC)** com IV aleatório de 128 bits (formato `enc1.<base64>.<hmac>`) para proteger snapshots em repouso + TLS 1.3 em trânsito.
18	**Dependências Seguras**	Versões mínimas seguras fixadas em `requirements.txt` e eliminação de pacotes legados desnecessários.
19	**Backup Automático**	Geração automática diária (janela de 24h) de snapshot completo criptografado na tabela `backups_automaticos` + endpoints para geração manual e exportação de stream binário seguro no formato `.scrcbak` com autenticação HMAC e hash `SHA-256`.
20	**Monitoramento & Auditoria Dinâmica**	Auditoria dinâmica em tempo real (avaliando HTTPS, CORS, secrets, RLS, backups e headers) exposta em `GET /api/seguranca/painel`, acompanhada de telemetria contínua de eventos classificados por severidade (`INFO`, `WARN`, `CRITICAL`).

6. Catálogo de Endpoints da API RESTful

REFERÊNCIA DA API

Método	Rota	Permissão Exigida	Descrição Técnica
`POST`	`/api/auth/login`	Público (Rate Limited)	Autentica usuário, emite token HMAC e cookies HttpOnly.
`POST`	`/api/auth/logout`	Autenticado	Encerra sessão e limpa cookies seguros.
`GET`	`/api/auth/me`	Autenticado	Retorna perfil e permissões do usuário logado.
`POST`	`/api/auth/alterar-senha`	Autenticado	Altera a própria senha validando os 5 critérios de força.
`GET/POST`	`/api/usuarios`	Admin (`requer_admin`)	Lista todos os usuários ou cria novo operador/admin.
`PUT/DEL`	`/api/usuarios/{id}`	Admin (`requer_admin`)	Atualiza permissões/senha, desbloqueia ou remove conta.
`GET`	`/api/seguranca/painel`	Admin (`requer_admin`)	Diagnóstico das 20 verificações, RLS, logs e backups.
`POST`	`/api/seguranca/backup`	Admin (`requer_admin`)	Gera snapshot criptografado (CTR-HMAC-SHA256) imediato.
`GET`	`/api/seguranca/backup/exportar`	Admin (`requer_admin`)	Exporta arquivo JSON completo com checksum SHA-256.
`GET/POST`	`/api/clientes`	`entrada` / `recebimento`	Lista e cadastra clientes.
`POST`	`/api/lotes/entrada`	`entrada`	Cria lote (e cliente opcional) gerando os cabeçotes em `TRIAGEM` .
`GET`	`/api/lotes` e `/api/lotes/{id}`	Autenticado	Consulta lotes, peças vinculadas e rastreabilidade.
`PUT`	`/api/cabecotes/{id}/checklist`	`recebimento`	Salva flags de rota da peça e envia para `LAVAGEM` .
`POST`	`/api/etapas/{id}/<etapa>`	Permissão da Etapa	Registra apontamento de Lavagem, Jateamento, Estanqueidade, Faceamento, Lapidação, Montagem, Qualidade ou Pintura.
`GET`	`/api/etapas/relatorios/lista`	`relatorios` / `qualidade`	Lista todos os cabeçotes com status do laudo técnico.
`GET`	`/api/etapas/{id}/relatorio-individual`	`relatorios` / `qualidade`	Retorna (ou constrói) o JSON completo do Relatório PDF.
`POST`	`/api/lotes/{id}/expedir`	`expedicao`	Registra saída com foto, NF, veículo, motorista e libera pallet.
`POST`	`/api/upload-foto`	Autenticado (Rate Limited)	Valida Magic Bytes da imagem e salva em `fotos_armazenadas` .

7. Variáveis de Ambiente, Execução Local e Deploy

DEVOPS & OPERAÇÃO

7.1. Variáveis de Ambiente Suportadas

- `DATABASE_URL`: String de conexão PostgreSQL do Supabase (ex.: `postgresql+pg8000://postgres.<ref>:<senha>@aws-1-sa-east-1.pooler.supabase.com:6543/postgres`). Se omitida localmente, utiliza SQLite (`scrc.db`).
- `SCRC_AUTH_SECRET`: Segredo mestre para derivação da chave HMAC-SHA256 de tokens de sessão.
- `SCRC_ENCRYPTION_KEY`: Segredo mestre para derivação da chave de criptografia em repouso (CTR-HMAC-SHA256).
- `SCRC_CORS_ORIGINS`: Lista separada por vírgulas de domínios adicionais autorizados no CORS.
- `SCRC_ENABLE_DOCS`: Defina como `1` apenas em ambiente de desenvolvimento caso deseje habilitar `/docs` (Swagger UI).

7.2. Comandos de Execução e Deploy Contínuo

```
# 1. Executar servidor localmente (Desenvolvimento)
cd backend
.\venv\Scripts\python.exe -m uvicorn app.main:app --host 0.0.0.0 --port 8080 --reload

# 2. Deploy em Produção (GitHub -> Vercel)
git add -A
git commit -m "chore: atualizacao do sistema"
git push origin main
```